Projet PHP : Web Service REST ou Site Web CRUD MVC

📋 Informations générales

Formation : UC D43 **Période :** Février - Avril **Modalité :** Projet individuel **Référente :** Morgane Flamant

---

🎯 Objectif du projet

Vous devez concevoir et développer **au choix** :

- **Option A :** Un web service REST complet
- **Option B :** Un site web CRUD en architecture MVC

Le projet doit utiliser la programmation orientée objet PHP avec un framework moderne (Symfony ou Laravel) et intégrer MongoDB pour la gestion de données NoSQL.

Exigences minimales obligatoires

Pour l'Option A - Web Service REST

- **Minimum 5 ressources** différentes avec relations entre elles
- **Au moins 15 endpoints** fonctionnels (CRUD complet sur plusieurs ressources)
- **Authentification JWT** ou OAuth2
- **Gestion des rôles** (admin, utilisateur, invité)
- **Pagination, filtrage et tri** sur les listes
- **Validation des données** avec messages d'erreur explicites
- **Codes HTTP appropriés** (200, 201, 400, 401, 403, 404, 500)
- **Documentation API** interactive (Swagger/OpenAPI ou équivalent)
- **Tests unitaires** : minimum 20 tests
- **Gestion des erreurs** centralisée

Pour l'Option B - Site Web CRUD MVC

- **Minimum 4 entités** métier avec relations
- **Interface utilisateur complète** : création, lecture, modification, suppression
- **Authentification et autorisation** (sessions sécurisées)
- **Gestion des rôles** avec restrictions d'accès
- **Formulaires avec validation** côté serveur et client
- **Recherche et filtres** avancés
- **Upload de fichiers** (images, documents)
- **Interface responsive** (Bootstrap, Tailwind ou équivalent)
- **Programmation asynchrone** : au moins 3 fonctionnalités AJAX/Fetch
- **Tests fonctionnels** : minimum 15 tests

📚 Compétences techniques à mobiliser

Langage et Programmation

- PHP Orienté Objet (POO)
- Principes SOLID et design patterns (ORM Doctrine, ODM)
- Modélisation UML2 de votre architecture

Frameworks

- **Symfony** ou **Laravel** (au choix)
- Architecture MVC

Bases de données

- **MongoDB** : base NoSQL orientée documents JSON
  - **Minimum 4 collections** avec relations (références ou embedded documents)
  - **Requêtes complexes** : agrégations, lookups, projections
  - **Map Reduce** : au moins 2 opérations fonctionnelles
  - **Index** : optimisation des requêtes fréquentes
  - **Scripts de Sharding** : configuration et démonstration
  - **Clustering** : configuration replica set (documentation)
- **Bases relationnelles** (optionnel pour données complémentaires) : PDO/MYSQLI

Web Services

- **REST** : API avec gestion complète des verbes HTTP (GET, POST, PUT, PATCH, DELETE)
- **HATEOAS** : liens hypermedia dans les réponses (recommandé)
- **Versioning API** : gestion des versions (v1, v2)
- **Rate limiting** : limitation du nombre de requêtes
- **CORS** : configuration pour appels cross-origin
- **SOAP** (optionnel pour bonus)
- Échanges de données : XML, JSON
- **CURL** : client pour consommer des API externes (au moins 1 intégration)

Technologies front-end

- **Programmation asynchrone** : AJAX et Fetch (minimum 3 fonctionnalités)
- **Manipulation de données** : JSON/XML, DTD
- **Interface dynamique** : mise à jour sans rechargement de page
- **Gestion des erreurs** côté client avec feedback utilisateur

---

📦 Livrables attendus

1. Code source

- **Dépôt Git** avec historique de commits structuré
- **Architecture MVC** strictement respectée
- **Code commenté** et respectant les conventions PSR-1, PSR-4, PSR-12

- **Design patterns** : au moins 3 patterns différents implémentés (Factory, Repository, Strategy, Observer, etc.)
- **Variables d'environnement** : fichier .env pour la configuration
- **Gestion des dépendances** : composer.json complet

2. Base de données MongoDB

- **Schéma de données** documenté avec justification des choix (embedded vs referenced)
- **Scripts de création** et d'initialisation avec données de test (minimum 50 documents)
- **Scripts Map Reduce** : au moins 2 opérations complexes
- **Scripts de Sharding** : configuration et démonstration
- **Configuration clustering** : replica set documenté
- **Index** : création et justification des index

3. Documentation technique

- **README.md** : installation pas à pas, configuration, utilisation, troubleshooting
- **Diagrammes UML2** obligatoires :
  - Diagramme de classes (complet)
  - Diagramme de séquences (au moins 3 scénarios)
  - Diagramme de cas d'utilisation
  - Diagramme de composants (architecture)
- **Documentation API** : endpoints, paramètres, réponses, exemples (Swagger/Postman Collection)
- **Guide d'architecture** : explication des choix techniques et patterns utilisés

4. Rapport de projet

- **Introduction** : contexte métier, problématique, objectifs
- **Analyse des besoins** : spécifications fonctionnelles et techniques
- **Choix techniques justifiés** : framework, MongoDB vs SQL, architecture
- **Architecture détaillée** : diagrammes, patterns, structure du code
- **Implémentation** : fonctionnalités clés, extraits de code commentés
- **MongoDB** : modélisation, requêtes complexes, Map Reduce, Sharding
- **Sécurité** : mesures implémentées (authentification, validation, injections)
- **Tests** : stratégie de tests, couverture, résultats
- **Difficultés et solutions** : analyse réflexive
- **Perspectives** : améliorations futures, scalabilité
- **Conclusion** : bilan des compétences acquises

5. Présentation orale (20 min + 10 min questions)

- **Support de présentation** : slides clairs et professionnels
- **Démonstration live** : scénarios d'utilisation complets
- **Explication technique** : architecture, code, MongoDB
- **Démonstration MongoDB** : requêtes complexes, Map Reduce
- **Tests en direct** : API avec Postman ou interface utilisateur
- **Retour d'expérience** : apprentissages, difficultés surmontées

---

💡 Suggestions de sujets (complexité suffisante requise)

Vous êtes libre de choisir votre thématique, mais elle doit permettre de démontrer toutes les compétences requises.

Option REST API (exemples)

- **API de gestion de projets** (type Trello) : projets, tâches, utilisateurs, commentaires, fichiers
- **API e-commerce** : produits, catégories, commandes, clients, paiements, avis
- **API de gestion d'événements** : événements, participants, réservations, lieux, organisateurs
- **API de plateforme éducative** : cours, modules, étudiants, évaluations, progression
- **API de réseau social** : utilisateurs, posts, commentaires, likes, messages, notifications

Option Site CRUD MVC (exemples)

- **Plateforme de gestion de formations** : formations, sessions, formateurs, apprenants, évaluations
- **Application de gestion de stock** : produits, fournisseurs, commandes, inventaire, alertes
- **Système de réservation** : ressources, réservations, utilisateurs, calendrier, paiements
- **Plateforme de location** : biens, propriétaires, locataires, contrats, paiements
- **Système de gestion RH** : employés, départements, congés, évaluations, documents

⚠ Sujets à éviter (trop simples)

- Simple blog
- To-do list basique
- Carnet d'adresses
- Gestionnaire de notes

✅ Critères d'évaluation détaillés

1. Architecture et Qualité du Code (40 points)

- **Architecture MVC** : séparation stricte des responsabilités (8 pts)
- **POO avancée** : SOLID, design patterns (minimum 3), héritage, interfaces (10 pts)
- **Framework** : utilisation appropriée, respect des conventions (6 pts)
- **Sécurité** : authentification, autorisation, validation, protection injections (8 pts)
- **Gestion des erreurs** : exceptions, logs, messages explicites (4 pts)
- **Code quality** : PSR, commentaires, lisibilité, pas de code dupliqué (4 pts)

2. MongoDB et Gestion des Données (20 points)

- **Modélisation** : schéma pertinent, relations, choix embedded/referenced justifiés (5 pts)
- **Requêtes complexes** : agrégations, lookups, projections variées (4 pts)
- **Map Reduce** : au moins 2 opérations fonctionnelles et pertinentes (5 pts)
- **Optimisation** : index, performances (2 pts)
- **Sharding/Clustering** : scripts et configuration documentés (4 pts)

3. Fonctionnalités et Services Web (20 points)

- **Complétude** : toutes les fonctionnalités CRUD implémentées (6 pts)
- **API REST** : verbes HTTP, codes statut, structure cohérente (5 pts)
- **Fonctionnalités avancées** : pagination, filtres, tri, recherche (4 pts)
- **Asynchrone** : AJAX/Fetch fonctionnels (3 pts)
- **Intégration externe** : CURL, consommation API tierce (2 pts)

4. Tests (10 points)

- **Tests unitaires** : couverture minimale 20 tests (5 pts)
- **Tests fonctionnels** : scénarios complets (3 pts)
- **Qualité des tests** : pertinence, assertions, cas limites (2 pts)

5. Documentation et UML (10 points)

- **README** : installation claire, configuration, utilisation (2 pts)
- **Diagrammes UML** : classes, séquences, cas d'utilisation, composants (4 pts)
- **Documentation API** : complète et exploitable (2 pts)
- **Rapport** : qualité, analyse, réflexion (2 pts)

6. Professionnalisme et Présentation (10 points)

- **Git** : commits structurés, messages clairs, branches (3 pts)
- **Présentation orale** : clarté, maîtrise du sujet (3 pts)
- **Démonstration** : fonctionnelle et convaincante (3 pts)
- **Respect des délais** : jalons et livraison finale (1 pt)

Bonus (10 points)

- Fonctionnalités supplémentaires innovantes (3 pts)
- SOAP implémenté (2 pts)
- Tests avec couverture > 80% (2 pts)
- CI/CD configuré (2 pts)
- Docker/containerisation (1 pt)

Total : 110 points (100 + 10 bonus) Seuil de validation : 50/100

🔧 Prérequis techniques

- PHP 8.x
- Composer
- MongoDB 6.x ou supérieur
- Git
- Symfony ou Laravel (dernière version stable)
- Postman ou équivalent (pour tester l'API)

📖 Ressources recommandées

Documentation officielle

- PHP.net
- Symfony ou Laravel
- MongoDB PHP
- Doctrine ODM

Outils

- PHPStorm ou VS Code
- MongoDB Compass
- Postman / Insomnia
- Git / GitHub / GitLab

⚠️ Attendus professionnels

- **Autonomie** : Rechercher des solutions, consulter la documentation
- **Rigueur** : Code propre, conventions respectées, tests réguliers
- **Gestion du temps** : Respecter les jalons et l'échéance finale
- **Documentation** : Produire une documentation exploitable
- **Versioning** : Utiliser Git de manière professionnelle
- **Proactivité** : Solliciter de l'aide en cas de blocage

🚫 Critères d'échec automatique

- Application non fonctionnelle ou incomplète (< 50% des fonctionnalités)
- Absence de MongoDB ou utilisation superficielle
- Pas d'architecture MVC
- Pas de framework utilisé
- Absence de Map Reduce
- Pas de tests
- Plagiat ou code copié sans compréhension
- Dépôt Git vide ou avec moins de 10 commits
- Absence de documentation

Bon courage pour ce projet ! 🚀